chapter_08/01/app/views/concerts/_search_result.html.erb

```
<article class="my-6 max-w-screen-lg">
  <div>
    <%= concert.start_time.by_example("Jan 2 @3:04 PM") %>
    <div class="font-bold text-xl">
      <%= link_to(concert, data: {"turbo-frame": "_top"}) do %>
        <%= highlight(concert.name, /#{query}/i) %>
      <% end %>
      <div class="float-right">
        <% if concert.sold_out? %>
          Sold out
        <% else %>
          <%= pluralize(concert.unsold_ticket_count, "Tickets") %>
          Remaining
        <% end %>
      </div>
    </div>
    <div>
      <%= highlight(concert.bands.map(&:name).join(", "), /#{query}/i) %>
    </div>
    <div><%= concert.genre_tags.split(",").to_sentence %></div>
    <div><%= concert.venue.name %></div>
  </div>
</article>
```

I want to point out here that we have a functional, if not fancy, Rails search. If you go to localhost:3000/concerts, you'll see an index page. If you go to localhost:3000/concerts?query=rock, you'll see search results. They'll look a little odd because we've styled them for a modal display, but they're not that bad. My point is that we can think about this as a progressive enhancement. We might have started with search being on a completely different page, but we've decided to make it part of the existing page for a better user experience.

When dealing with Hotwire functionality, it is almost always a good idea to get a server-side only version of the functionality working before starting to add client-side interaction. Doing the work this way will often result in a simpler final design, and you may find ways to do things in plain Turbo that you thought would require custom JavaScript.

Let's make that client-side interaction and modal work. First, we need to put a